

REPUBLIQUE DU CAMEROUN

Paix-Travail-Patrie

MINISTERE DE
L'ENSEIGNEMENT
SUPERIEUR

Université de Yaoundé 1

Saint Jean Ingénieur



INSTITUT UNIVERSITAIRE
SAINT JEAN

REPUBLIC OF
CAMEROON

Peace-Work-Fatherland

MINISTRY OF HIGHER
EDUCATION

University of Yaoundé 1

Saint Jean Engineer

RAPPORT DE L'UE Théorie des Systèmes d'exploitation

THEME:

Réalisation de la commande mylsof

- BAYI NGATCHEU ANGE FRANKA
- KENGNE YAKOUMBO ANGE NAOMY
- NTCHANTCHOU MGASSA SOBRIN
- SARR SALIF
- TIENTCHEU KAMGA ORTHYS CASSANDRA

Etudiant en **Troisième année du cycle ingénieur en informatique**

Sous l'encadrement de

Mr NGUIMBUS

Enseignant à l'IUSJC

ANNEE
ACADEMIQUE
2025-2026

SOMMAIRE

INTRODUCTION	3
I. ARCHITECTURE DU PROJET	4
II. PRÉSENTATION DES OPTIONS IMPLEMENTÉES.....	5
1. Option opt_Cassandra.....	5
a) Bibliothèques et fonctions utilisées :	5
b) Récapitulatifs des 09 fonctions	5
c) Présentation et implémentation des fonctions.....	5
2. Option opt_Naomy.....	7
a) Récapitulatifs des 09 fonctions	7
b) Présentations des fonctions	7
3. Option opt_Ange.....	8
a) Récapitulatifs des 09 fonctions	8
b) Présentation des fonctions.....	9
4. Option opt_Salif.....	11
a) Récapitulatif des 09 fonctions.....	11
b) Présentation des fonctions.....	11
5. Option opt_Sobrin.....	13
a) Récapitulatif des 09 fonctions.....	13
b) Présentation des fonctions.....	14
• Option -z : option_z().....	15
III. DIFFICULTÉS RENCONTRÉES.....	18
CONCLUSION.....	19

INSTITUT SAINT JEAN

INTRODUCTION

Dans le cadre de la validation de l'unité d'enseignement « Théorie des Systèmes d'exploitation », nous avons été amenés à confronter les concepts théoriques abordés en cours à la réalité pratique du développement système. L'objectif principal de ce projet de fin de semestre était de dépasser la simple utilisation des outils standards pour en comprendre le fonctionnement interne. Pour ce faire, il nous a été demandé de concevoir et de développer from scratch une réplique fonctionnelle d'une commande essentielle de l'écosystème Unix/Linux : lsof (List Open Files), que nous avons baptisée mylsof.

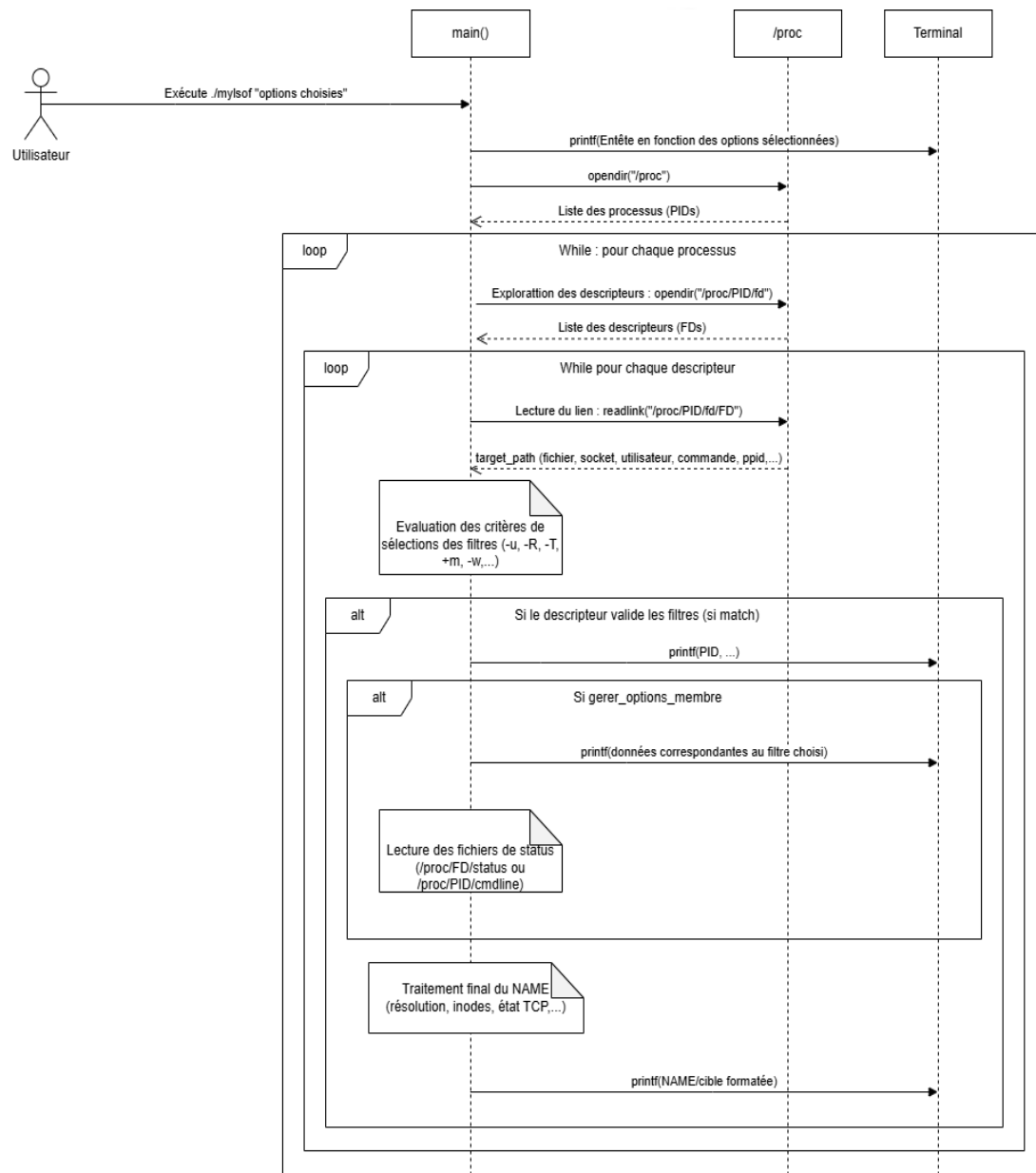
Sous les systèmes de type Unix, et plus particulièrement sous Linux, la philosophie fondamentale est que « tout est fichier ». Cette abstraction unifie l'accès aux ressources : que ce soit des documents texte sur le disque dur, des périphériques matériels comme le clavier ou l'écran, ou encore des canaux de communication réseau comme les sockets, tout est manipulé via le même paradigme des fichiers. La commande lsof est un outil d'administration système puissant qui permet d'interroger le noyau pour révéler quels processus maintiennent quels fichiers ouverts. C'est un instrument indispensable pour le diagnostic, le débogage et la supervision de l'activité système en temps réel.

Le défi technique de ce projet résidait dans l'interdiction d'utiliser la commande lsof existante ou des bibliothèques de haut niveau qui auraient masqué la complexité sous-jacente. Notre programme devait interagir directement avec le noyau Linux. Pour ce faire, nous nous sommes appuyés sur le système de fichiers virtuel /proc. Ce système, monté par défaut sur la racine /proc, ne stocke pas de données persistantes sur le disque, mais expose dynamiquement l'état du noyau et des processus en cours d'exécution sous forme de fichiers texte et de répertoires.

Le développement de mylsof s'est articulé autour d'une architecture modulaire en langage C, répartie entre les cinq membres de l'équipe : Cassandra, Naomy, Ange, Salif et Sobrin. Chaque membre a été responsable de l'implémentation d'un ensemble de neuf options spécifiques, couvrant des fonctionnalités variées telles que le filtrage par utilisateurs, par processus, l'analyse des connexions réseau, ou encore l'examen de la mémoire mappée. L'approche « from scratch » nous a contraints à maîtriser des concepts de bas niveau : la navigation dans l'arborescence /proc, la lecture de liens symboliques avec readlink, l'extraction d'informations via stat, et une gestion rigoureuse de la mémoire et des chaînes de caractères en C. Ce rapport présente la méthodologie adoptée, l'architecture détaillée du projet, ainsi que la description technique des quarante-cinq options implémentées par notre équipe de cinq membres.

I. ARCHITECTURE DU PROJET

Le projet est divisé en plusieurs modules, chacun géré par un étudiant. Chaque module correspond à un fichier source C contenant une série de 9 options spécifiques. L'ensemble de ces fichiers est ensuite compilé et lié pour former l'exécutable final **mylsof**. L'architecture repose sur la lecture des pseudo-fichiers présents dans le répertoire **/proc**, qui contiennent des informations dynamiques sur le système et les processus en cours d'exécution.



II. PRÉSENTATION DES OPTIONS IMPLÉMENTÉES

1. Option opt_Cassandra

Cette partie présente les 09 fonctions implémentées par Cassandra dans le fichier `opt_Cassandra.c`, toutes ces fonctions sont développées from scratch, c'est-à-dire en utilisant uniquement les bibliothèques standard du langage C sans appel à la vraie commande `ls`.

a) Bibliothèques et fonctions utilisées :

Les différentes bibliothèque et fonctions utilisées sont les suivantes :

- Bibliothèque `stdio.h` : `printf`, `fprintf`, `fopen`, `fgets`, `fclose`, `perror`, `scanf`
- Bibliothèque `stdlib.h` : `atoi`, `memset`
- Bibliothèque `string.h` : `strcmp`, `strncmp`, `strlen`, `snprintf`
- Bibliothèque `pwd.h` : `getpwnam`
- Bibliothèque `dirent.h` : `opendir`, `readdir`, `closedir`
- Bibliothèque `unistd.h` : `readlink`, `dup2`, `fileno`

b) Récapitulatifs des 09 fonctions

Option	Nom de la fonction	Description courte
-d	Option_d(char *dir)	Filtre par numéro de file descriptor
-u	Option_u(char *user)	Filtre par nom d'utilisateur
-l	Option_l(void)	Afficher les UIDs numériques
-F	Option_F(char *fields)	Format parseable pour scripts
-S	Option_S(int sec)	Définir le timeout (min 2s)
-D	Option_D(char *dir)	Cherche dans un répertoire
+/-e	Option_e(int active)	Ignorer/signaler erreurs filesystem
-k	Option_k(void)	Lire le fichier kernel map
+/-w	Option_w(int active)	Active/désactive les warnings

c) Présentation et implémentation des fonctions

- **Option -d : option_d (char dir) :** L'option -d permet de filtrer des fichiers ouverts par numéros de file descriptor. Un file descriptor (fd) est un numéro que linux attribue à chaque fichier ouvert par un processus : 0 = entrée clavier, 1 = sortie écran, 2 = erreur. La fonction parcourt tous les processus dans `/proc` et n'affiche que ceux dont le fd correspond aux numéros demandés.

Exemple d'appel : **option_d ("1")** Sortie attendue : PID 1 -> fd 1 -> /Dev/console
PID 491 -> fd -> /Dev/null PID 518 -> fd -> pipe : [1011]

- **Option -u : option_u (char user)** : L'option -u filtre les fichiers ouverts par nom d'utilisateur. La fonction convertit le nom en UID grâce à getpwnam (), puis parcourt/proc pour lire le fichier status de chaque processus. Elle compare l'UID réel du processus avec celui de l'utilisateur demandé, et affiche uniquement les fichiers correspondants.

Exemple d'appel : option_u("root") ; Sortie attendue : PID 1 -> fd 0 -> /dev/console PID 1 -> fd 1 -> /dev/console PID 1 -> fd2 -> /dev/console

- **Option -l : option_l(void)** : L'option -l affiche les UIDs numériques au lieu des noms d'utilisateurs. Normalement lsof affiche le nom (ex : Cassandra), avec -l il affiche le numéros (ex : 1001). La fonction lit /proc/[pid]/status pour chaque processus et extrait la ligne Uid : qui contient les identifiants numériques.

Exemple d'appel : option_l () ; Sortie attendue : PID 1 -> Uid : 0 0 0 0 PID 12 -> Uid : 0 0 0 0 PID 66 -> Uid : 0 0 0 0

- **Option -F : option_F(char fields)** : L'option -F affiche la sortie dans un format parseable, destiné aux scripts. Chaque information est préfixée par une lettre : p pour le PID, f pour le file descriptor, et n pour le nom du fichier. Ce format permet à d'autres programmes d'analyser facilement la sortie de mylsof.

Exemple d'appel : option_F ("pfn") ; Sortie attendue : p1 f0 n/dev/console f1 n/dev/console p12 f0 n/dev/null Cette sortie veut tout simplement dire que le processus 1 a ouvert /dev/console sur ses descripteurs 0 et 1 et le processus 12 a ouvert /dev/null sur son descripteur 0.

- **Option -S : option_S (int sec)** : L'option -S définit le timeout pour les appels système lents, notamment sur les systèmes de fichiers réseau (NFS). La valeur minimale est de 2 secondes. Si une valeur inférieure est fournie, la fonction la force automatiquement à 2 et affiche un message d'avertissement.

Exemple d'appel : option_S (1) : sera forcée à 2. Option_S (5) : acceptée. Sortie attendue : Timeout minimum : 2 secondes. Valeur forcée à 2 Timeout défini à 2 secondes Timeout défini à 5 secondes

- **Option -D : option_D (char dir)** : L'option -D liste tous les fichiers ouverts dont le chemin commence par le répertoire donné. La fonction parcourt tous les processus dans /proc, suit les liens symboliques dans /proc/[pid]/fd pour obtenir le chemin réel de chaque fichier, puis vérifie si ce chemin correspond au répertoire demandé.

*Exemple d'appel : void option_D (char *dir) Sortie attendue : PID 550 -> fd 3 -> /proc PID 550 -> fd 4 -> /proc/550/fd*

- **Option +/-e : option_e (int activer)** : L'option +/-e contrôle la gestion des erreurs sur un système de fichiers. Avec +e (activer=0) les erreurs sont ignorées (mode silencieux), utile pour les systèmes NFS lents. Avec -e (activer=1) les erreurs sont signalées.

Exemple d'appel : option_e (0) : qui veut dire +e et option_e (1) : qui veut dire -e. Sortie attendue : Mode +e : les erreurs file system seront ignorées Mode -e : les erreurs file system seront signalées

- **Option -k : option_k(void) :** L'option -k lit les noms de symboles du noyau linux depuis le fichier /boot/system.map. Ce fichier contient les adresses mémoire des fonctions internes du kernel. La fonction ouvre ce fichier, affiche les 20 premières lignes et indique s'il y en a davantage.
Exemple d'appel : option_k () ; Sortie attendue : Lecture du fichier kernel map :
ffffff81000000 T startup_64 fffffff81000000 T secondary_startup_64
- **Option +/-w : option_w(int active) :** L'option +/-w active ou désactive les messages d'avertissement. Avec +w (activer=0) stderr est redirigé vers /dev/null (la poubelle linux) supprimant tous les warnings. Avec -w (activer=1) stderr est restauré vers /dev/tty. La fonction utilise dup2() pour effectuer la redirection réelle.
Exemple d'appel : option_w (0) ; pour +w et option_w (1) pour -w. Sortie attendue : Mode +w : tous les avertissements sont désactivés et Mode -w : tous les avertissements sont actifs.
- **Fonction gerer_options_Cassandra :** La fonction gerer_option_Cassandra () est le point d'entrée principal du fichier opt_Cassandra.c. Elle reçoit les arguments de la ligne de commande et appelle la bonne fonction selon l'option détectée. Elle retourne 1 si l'option est reconnue et 0 sinon.

2. Option opt_Naomy

Cette partie présente les 09 fonctions implémentées par Naomy dans le fichier opt_Naomy.c, toutes ces fonctions sont développées from scratch, c'est-à-dire en utilisant uniquement les bibliothèques standard du langage C sans appel à la vraie commande lsof.

a) Récapitulatifs des 09 fonctions

Option	Nom de la fonction	Description courte
-n	option_n	Exécution simulée
fichier	option_fichier	Spécifie un fichier en argument
-i	option_i	Ignore la case ou active le mode interactif
-R	option_R	Parcours récursif des dossiers
-b	option_b	Crée une sauvegarde
+/-m	option_plus_m	Active/désactive un mode
-V	option_V	Affiche la version ou mode verbeux
-T	option_T	Définit un type ou active un test
-x	option_x	Exclut des fichiers ou mode debug

b) Présentations des fonctions

- **Option -b : (avoid blocking)** elle ordonne à la commande lsof d'éviter les appels systèmes bloquants comme stat, lstat ou readlink et renvoie la liste des fichiers ouverts sans que la commande ne se fige.

- **Option -n : (no host names)** elle empêche lsof de convertir les adresses IP en noms de domaine ; autrement dit, elle désactive la résolution de noms de domaine DNS et affiche uniquement les adresses IP numériques pour les connexions réseaux.
- **Option -R : (parent process ID)** elle demande à afficher l'identifiant du processus parent du processus en cours (les processus étant considéré comme des fichiers sur linux) en ajoutant une colonne.
- **Option -i : (internet sockets)** elle isole uniquement les fichiers liés aux connexions réseau (IPV4 et IPV6) tels que les ports TCP/UDP ouverts, les connexions internet établies ou les sockets réseaux.
- **Option fichier :** On passe le nom d'un fichier ou son chemin d'accès en argument, pour renvoyer la liste de tous les processus qui l'ont ouvert.
- **Option -T : (TCP/TPI information)** elle contrôle l'affichage des informations détaillées des connexions réseau TCP.
- **Option -x : (cross-référence/symlinks)** elle modifie le comportement de lsof lorsqu'il doit inspecter le contenu d'un dossier.
- **Option -V : (verbose/explanations)** elle demande à la commande lsof de renvoyer des messages d'avertissement ou d'explication à l'écran lorsqu'un filtre n'a pas de résultats ou qu'on n'a pas accès à un fichier ou dossier.
- **Option +m : (mount supplement)** elle sert à obtenir et renvoyer les numéros d'identification des disques lorsque le noyau refuse de les donner via des requêtes standards comme stat.

3. Option opt_Ange

Cette partie présente les 09 fonctions implémentées par Ange dans le fichier opt_Ange.c. Toutes ces fonctions sont développées from scratch, c'est-à-dire en utilisant uniquement les bibliothèques standard du langage C et les appels système, sans aucun recours à la commande lsof native.

a) Récapitulatifs des 09 fonctions

Option	Nom de la fonction	Description courte	Élément principal utilisé
-p	option_p(char *pid_list)	Filtre les fichiers ouverts par un ou plusieurs PID	/proc/[PID]/fd/
-c	option_c(char *comm)	Filtre par nom de commande (processus)	/proc/[PID]/comm
+d	option_plus_d(char *dir)	Liste les fichiers ouverts dans un répertoire (non récursif)	readlink() et dirname()
-r	option_r(int sec)	Mode répétition (rafraîchissement toutes les x secondes)	Boucle while et sleep()
-C	option_C(void)	Désactive la table des descripteurs du noyau (optimisation CPU)	Indicateur global

+/-E	option_E(int active)	Active/désactive l'affichage des endpoints (pipes/sockets)	Extraction d'ID d'inodes
-A	option_A(void)	Sélectionne uniquement les fichiers sous verrous système (locks)	/proc/locks
-v	option_v(void)	Mode verbeux (affiche la version et des statistiques de scan)	printf()
+/-M	option_M(int active)	Active/désactive le mapping mémoire des fichiers (maps)	/proc/[PID]/maps

b) Présentation des fonctions

• Option -p : option_p(char *pid_list)

Rôle : Permet de cibler un processus spécifique en fournissant son identifiant (PID) afin de lister uniquement ses fichiers ouverts.

Logique d'implémentation : Au lieu de parcourir l'intégralité du répertoire /proc, la fonction isole le sous-répertoire /proc/[PID]/fd/ correspondant au PID passé en paramètre. Elle ouvre ce dossier via opendir(), lit les descripteurs de fichiers avec readdir() et résout leurs chemins absolus en utilisant l'appel système readlink().

• Option -c : option_c(char *comm)

Rôle : Filtre la liste des fichiers ouverts selon le nom de la commande ou de l'exécutable qui a lancé le processus.

Logique d'implémentation : Le programme parcourt les dossiers numériques de /proc. Pour chaque PID, il ouvre le fichier /proc/[PID]/comm qui contient le nom brut de l'exécutable. À l'aide de la fonction strncmp(), il compare ce nom avec la chaîne comm fournie par l'utilisateur. Si la correspondance est validée, les fichiers du processus sont extraits.

• Option +d : option_plus_d(char *dir)

Rôle : Recherche et liste les fichiers ouverts se trouvant strictement dans le répertoire spécifié en argument, de manière non récursive.

Logique d'implémentation : La fonction convertit d'abord le chemin fourni en chemin absolu stable via realpath(). Lors du scan des liens symboliques dans /proc/[PID]/fd/, elle récupère la destination réelle du descripteur. Elle utilise ensuite la fonction dirname() sur ce chemin pour obtenir le dossier parent et vérifie avec strcmp() s'il est identique au répertoire cible.

• Option -r : option_r(int sec)

Rôle : Passe la commande mylsof en mode de surveillance continue, en répétant l'affichage à intervalles réguliers de temps.

Logique d'implémentation : Cette option encapsule le cycle principal de détection et d'affichage dans une boucle contrôlée while. Après avoir parcouru le système /proc et affiché les résultats sur le terminal, la fonction suspend l'exécution du programme pendant le nombre de secondes requis à l'aide de l'appel système sleep(sec) avant de vider l'écran et de relancer un scan.

- **Option -C : option_C(void)**

Rôle : Désactive ou contourne la lecture de certaines structures du noyau pour économiser l'utilisation du processeur (optimisation CPU) lors de grands scans.

Logique d'implémentation : Elle configure un drapeau d'état booléen global. Lorsque ce mode est actif, le programme réduit ses accès aux fichiers secondaires de /proc (comme le parsing répété de comm ou de fichiers de configuration réseau), limitant ainsi le nombre d'appels système gourmands pour accélérer l'exécution.

- **Option +/-E : option_E(int active)**

Rôle : Contrôle (+E pour activer, -E pour désactiver) l'affichage détaillé des liaisons d'endpoints pour les canaux de communication inter-processus (Pipes et Sockets).

Logique d'implémentation : Si l'option est active et que readlink() renvoie une chaîne identifiée comme un tube (pipe:[inode]) ou un socket, la fonction marque graphiquement le descripteur et cherche à faire correspondre l'identifiant unique de l'inode avec les autres processus lisant ou écrivant dans ce même canal pour exposer la liaison.

- **Option -A : option_A(void)**

Rôle : Restreint l'affichage de mylsof aux seuls fichiers qui font actuellement l'objet d'un verrouillage système actif (locks).

Logique d'implémentation : La fonction ouvre et analyse en premier lieu le pseudo-fichier /proc/locks, qui expose tous les verrous (POSIX, FLOCK) gérés par le noyau Linux. Elle extrait les numéros d'inodes des fichiers verrouillés, puis applique un filtre vu lors du parcours des descripteurs pour ne retenir que les fichiers présents dans cette liste.

- **Option -v : option_v(void)**

Rôle : Mode verbeux qui affiche les informations de version du programme ainsi que des statistiques de traitement sur le terminal.

Logique d'implémentation : Elle utilise printf() pour afficher les en-têtes contenant la version actuelle du projet (2023-2024). De plus, elle incrémente des compteurs internes pendant l'exécution pour afficher en fin de rapport des indicateurs comme le nombre total de répertoires de processus scannés et le nombre d'erreurs de permissions rencontrées.

- **Option +/-M : option_M(int active)**

Rôle : Active (+M) ou désactive (-M) l'exploration et l'affichage des fichiers mappés en mémoire par les processus (fichiers de bibliothèques .so, segments de texte).

Logique d'implémentation : Lorsque l'option est activée, en plus de lire le dossier /proc/[PID]/fd/, la fonction ouvre et extrait les données du pseudo-fichier de cartographie mémoire /proc/[PID]/maps. Elle lit ce fichier ligne par ligne à l'aide de fgets(), repère les chemins absolus des fichiers chargés en mémoire virtuelle et les ajoute à la liste des fichiers ouverts par le processus.

• **Fonction de gestion : gerer_options_Ange(int argc, char *argv[])**

Rôle : Point d'entrée logique du module opt_Ange.c chargé d'analyser la ligne de commande et d'activer les traitements requis.

Logique d'implémentation : Elle examine séquentiellement le tableau d'arguments argv. Dès qu'un commutateur de son bloc de 9 options est intercepté, elle configure les variables globales du module ou extrait les arguments requis (comme la valeur en secondes pour -r ou la chaîne pour -p). Elle retourne 1 si l'option analysée fait partie de ses attributions, et 0 sinon.

4. Option opt_Salif

Cette partie présente les 09 fonctions implémentées par Salif dans le fichier opt_salif.c, toutes ces fonctions sont développées from scratch, c'est-à-dire en utilisant uniquement les bibliothèques standard du langage C sans appel à la vraie commande lsof.

a) Récapitulatif des 09 fonctions

Option	Nom de la fonction	Description courte
-P	Option_P	Afficher les sockets ouverts par processus
-g	Option_g	Filtrer les processus selon leur GID
-U	Option_U	Afficher les sockets Unix
-s	Option_s	Afficher la taille des fichiers ouverts
-X	Option_X	Afficher les fichiers mappés en mémoire
-Z	Option_Z	Afficher le contexte SELinux
-N	Option_N	Afficher les fichiers NFS
-j	Option_j	Afficher le namespace de montage
-c	Option_c	Filtrer par nom de processus

b) Présentation des fonctions

- **Option -P :** L'option -P permet d'afficher les sockets ouverts par les processus. Le programme commence par ouvrir le dossier /proc, puis il parcourt tous les processus. Pour chaque processus, il lit son nom dans /proc/PID/comm. Ensuite, il ouvre le dossier /proc/PID/fd, qui contient les descripteurs de fichiers ouverts par le processus. Chaque descripteur est un lien symbolique. Le programme utilise donc readlink() pour

connaître la ressource vers laquelle pointe le descripteur. Si le lien commence par : socket, il est considéré comme tel.

- **Option -g** : L'option -g permet de filtrer les processus selon leur groupe. Elle reçoit un GID en argument. Par exemple : `./mysof -g 1000`. Le programme parcourt les processus dans `/proc`, puis lit le fichier : `/proc/PID/status`. Dans ce fichier, il recherche la ligne qui commence par : `Gid`. Cette ligne contient les identifiants de groupe du processus. Le programme extrait le GID avec `sscanf()`, puis le compare avec le GID donné par l'utilisateur. Si les deux valeurs correspondent, le processus est affiché avec son nom, son PID et son GID.
- **Option -U** : L'option -U sert à afficher les sockets Unix. Dans cette implémentation, elle parcourt tous les processus, lit leur nom dans `/proc/PID/comm`, puis ouvre `/proc/PID/fd` pour analyser les fichiers ouverts. Comme pour l'option -P, elle utilise `readlink()` pour connaître la destination de chaque descripteur. Si le lien commence par : `socket:[...]`, il est affiché.
- **Option -s** : L'option -s permet d'afficher la taille des fichiers ouverts. Le programme parcourt les processus, récupère leurs fichiers ouverts dans `/proc/PID/fd`, puis utilise `readlink()` pour connaître le chemin du fichier. Ensuite, il appelle la fonction `stat(lien, &info)`. La fonction `stat()` permet d'obtenir des informations sur un fichier, comme sa taille. Dans cette option, la donnée utilisée est `info.st_size`. L'option affiche donc le nom du processus, le PID, la taille du fichier et le chemin du fichier.
- **Option -X** : L'option -X permet d'afficher les fichiers mappés en mémoire. Contrairement à plusieurs autres options, elle ne se base pas principalement sur `/proc/PID/fd`, mais sur le fichier : `/proc/PID/maps`. Ce fichier contient les zones mémoire utilisées par un processus. Le programme lit ce fichier ligne par ligne avec `fgets()`. Ensuite, il utilise `sscanf()` pour extraire l'adresse mémoire et le chemin du fichier. Si le chemin commence par `/`, cela signifie qu'il s'agit d'un vrai chemin de fichier.
- **Option -Z** : L'option -Z permet d'afficher le contexte SELinux des processus. SELinux est un mécanisme de sécurité sous Linux. Pour récupérer ce contexte, le programme lit le fichier : `/proc/PID/attr/current`. Si ce fichier est accessible, le programme lit le contexte SELinux du processus. Si le fichier n'est pas disponible, il affiche : (SELinux inactif).
- **Option -N** : L'option -N permet d'afficher les fichiers ouverts qui semblent appartenir à un système NFS. Dans cette implémentation, la détection est basée sur le chemin du fichier. Après avoir récupéré le lien du descripteur avec `readlink()`, le programme vérifie si le chemin commence par `/net/`, `/nfs/` ou s'il contient le mot `nfs`. Si une de ces conditions est vraie, le fichier est considéré comme un fichier NFS et il est affiché.
- **Option -j** : L'option -j permet d'afficher le namespace de montage des processus. Un namespace est un mécanisme Linux qui permet d'isoler certaines ressources du système. Le programme lit le lien symbolique : `/proc/PID/ns/mnt`. Ce lien peut donner une valeur comme : `mnt:[4026531840]`. Après avoir récupéré ce namespace avec `readlink()`, le programme parcourt les fichiers ouverts du processus et affiche le nom du processus, le PID, le descripteur, le namespace et le fichier ouvert.

- **Option -c** : L'option -c permet de filtrer les processus selon leur nom. Elle prend un nom en argument (ex: `./mylsof -c bash`). Le programme lit le nom de chaque processus dans : `/proc/PID/comm`. Ensuite, il compare le début du nom du processus avec le nom donné par l'utilisateur en utilisant `strncmp()`. Si le nom correspond, le programme affiche les fichiers ouverts par ce processus.
- **Gestion Des Options** : Toutes les options sont appelées dans une fonction principale qui analyse les arguments de la ligne de commande et redirige vers les fonctions spécifiques décrites ci-dessus.

5. Option opt Sobrin

Cette partie présente les 09 fonctions implémentées par Sobrin dans le fichier `opt_Sobrin.c`, toutes ces fonctions sont développées from scratch, c'est-à-dire en utilisant uniquement les bibliothèques standard du langage C sans appel à la vraie commande `ls`.

a) Récapitulatif des 09 fonctions

Option	Nom de la fonction	Objectif	Élément principal utilisé
-l	<code>option_l()</code>	Afficher les UIDs numériques au lieu des noms utilisateurs	<code>/proc/[pid]/status</code>
-h	<code>option_h()</code>	Afficher l'aide et les options disponibles	<code>printf()</code>
-o	<code>option_o()</code>	Afficher l'offset des fichiers ouverts	<code>/proc/[pid]/fdinfo/[fd]</code>
-z	<code>option_z()</code>	Afficher une colonne ZONE par compatibilité	Sortie formatée
-t	<code>option_t()</code>	Afficher uniquement les PID	Sortie simplifiée
-a	<code>option_a()</code>	Combiner les filtres en logique AND	Structure Options
+D	<code>option_plus_D(char *dir)</code>	Filtrer les fichiers ouverts dans un dossier et ses sous-dossiers	<code>realpath()</code> , <code>readlink()</code>

-k	option_k(char *file)	Accepter un fichier kernel map ou afficher un avertissement	access()
Gestion	gerer_options_MYLSOF()	Analyser les arguments et activer les bonnes options	argc, argv

b) Présentation des fonctions

- **Option -l : option_l()**

Rôle : L'option -l demande à MYLSOF d'afficher les identifiants utilisateurs numériques au lieu des noms d'utilisateurs. Sans cette option, le programme peut convertir l'UID en nom grâce à getpwuid(). Avec -l, il garde directement la valeur numérique.

Logique d'implémentation :

1. Pour chaque processus, la fonction ouvre le fichier /proc/[pid]/status.
2. Elle recherche la ligne qui commence par Uid:.
3. Elle extrait le premier UID, qui correspond à l'utilisateur réel du processus.
4. Au moment de l'affichage, elle écrit l'UID numérique dans la colonne USER.

Exemple d'appel :

```
./mysof -l
```

Sortie attendue :

COMMAND	PID	USER	FD	TYPE	SIZE/OFF	NAME
bash	1200	1000	0r	CHR	-	/dev/pts/0

Explication : Cette option est utile lorsque l'on veut éviter les conversions de noms ou lorsque certains utilisateurs n'existent pas dans la base locale du système.

- **Option -h : option_h()**

Rôle : L'option -h affiche une aide claire pour l'utilisateur. Elle explique le rôle de MYLSOF, la syntaxe générale de la commande et la liste des options disponibles.

Logique d'implémentation :

5. La fonction est appelée dès que l'argument -h, --help ou -? est détecté.
6. Elle affiche un texte d'aide avec printf().
7. Le programme s'arrête ensuite proprement avec un code de retour 0.
8. Aucun parcours de /proc n'est nécessaire dans ce cas.

Exemple d'appel :

```
./mysof -h
```

Sortie attendue :

MYLSOF - mini clone de lsof
 Utilisation : ./mylsof [options]
 -l afficher les UID numériques
 +D filtrer par dossier

Explication : Cette fonction améliore l'expérience utilisateur, car elle permet de comprendre rapidement comment utiliser le programme sans lire directement le code source.

- **Option -o : option_o()**

Rôle : L'option -o affiche l'offset associé à chaque descripteur de fichier. L'offset représente la position courante de lecture ou d'écriture dans un fichier ouvert.

Logique d'implémentation :

9. Pour chaque descripteur, le programme ouvre /proc/[pid]/fdinfo/[fd].
10. Il lit la ligne pos:, qui contient l'offset courant.
11. Il peut aussi lire la ligne flags: pour déterminer le mode d'ouverture du fd.
12. La colonne SIZE/OFF est remplacée par une colonne OFFSET lorsque -o est activée.

Exemple d'appel :

```
./mylsof -o
```

Sortie attendue :

COMMAND	PID	USER	FD	TYPE	OFFSET	NAME
code	2350	axel	12u	REG	0t456	
/home/axel/mylsof/main.c						

Explication : Cette option est surtout intéressante pour les fichiers réguliers. Pour certains objets comme les sockets ou les pipes, l'offset peut être absent ou non significatif.

- **Option -z : option_z()**

Rôle : L'option -z est liée à la notion de zones dans certaines versions de lsof, notamment sur Solaris. Dans un environnement Linux ou WSL, cette notion n'existe pas réellement, mais MYLSOF peut accepter l'option pour rester compatible avec la syntaxe.

Logique d'implémentation :

13. Lorsque -z est détectée, une colonne ZONE est ajoutée à la sortie.
14. Sous Linux/WSL, la valeur affichée peut être linux ou default.
15. L'option ne change pas la recherche des fichiers ouverts.
16. Elle agit uniquement sur le format d'affichage.

Exemple d'appel :

```
./mylsof -z
```

Sortie attendue :

COMMAND	PID	USER	FD	TYPE	SIZE/OFF	ZONE	NAME
sshd	700	root	3u	SOCK	-	linux	
socket:[12345]							

Explication : L'intérêt principal de cette option est d'éviter une erreur lorsque l'utilisateur saisit -z. Le programme reste donc plus proche de lsof, même si la fonctionnalité est simplifiée.

- **Option -t : option_t()**

Rôle : L'option -t affiche uniquement les PID des processus qui possèdent des fichiers ouverts correspondant aux critères. Elle est pensée pour être utilisée dans des scripts ou avec d'autres commandes.

Logique d'implémentation :

17. Le programme parcourt les processus comme d'habitude.
18. Dès qu'un processus possède au moins un fichier ouvert correspondant, seul son PID est affiché.
19. L'en-tête du tableau est supprimé afin de produire une sortie simple.
20. Pour éviter les doublons, un PID est affiché une seule fois par processus.

Exemple d'appel :

```
./mylsof -t +D /tmp
```

Sortie attendue :

```
1200
2350
3102
```

Explication : Cette option est très pratique pour chaîner les commandes, par exemple pour obtenir rapidement les processus utilisant un dossier donné.

- **Option -a : option_a()**

Rôle : L'option -a sert à combiner plusieurs critères de recherche avec une logique ET. Cela signifie que le résultat doit respecter tous les filtres demandés, et non seulement l'un d'entre eux.

Logique d'implémentation :

21. La présence de -a est mémorisée dans la structure Options.
22. Chaque filtre activé est ensuite évalué séparément.
23. Si -a est actif, un fichier ouvert doit valider tous les filtres pour être affiché.
24. Dans la version actuelle, l'option devient surtout utile lorsqu'elle est combinée avec +D, -o, -z ou de futurs filtres comme -u et -p.

Exemple d'appel :

```
./mylsof -a -o +D /tmp
```

Sortie attendue :

COMMAND	PID	USER	FD	TYPE	OFFSET	NAME
python3	3102	axe1	5u	REG	0t2048	/tmp/test.log

Explication : Cette fonction prépare une architecture propre et évolutive. Elle permettra d'ajouter facilement d'autres filtres tout en gardant un comportement cohérent.

- **Option +D : option_plus_D(char *dir)**

Rôle : L'option +D permet d'afficher uniquement les fichiers ouverts qui se trouvent dans un dossier précis ou dans l'un de ses sous-dossiers. C'est une option très utile pour savoir quels processus utilisent encore un répertoire.

Logique d'implémentation :

25. La fonction récupère le chemin donné après +D.
26. Elle utilise realpath() pour obtenir le chemin absolu du dossier.
27. Pour chaque fd, readlink() permet de connaître le fichier réellement ouvert.
28. Le programme compare ensuite le début du chemin du fichier avec le chemin du dossier demandé.
29. Si le chemin correspond, la ligne est affichée ; sinon elle est ignorée.

Exemple d'appel :

```
./mylsof +D /tmp
```

Sortie attendue :

COMMAND	PID	USER	FD	TYPE	SIZE/OFF	NAME
python3	3102	axel	5u	REG	1200	/tmp/test.log

Explication : Certains fichiers peuvent apparaître avec la mention (deleted). Le programme peut nettoyer cette mention avant de comparer les chemins.

- **Option -k : option_k(char *file)**

Rôle : L'option -k est prévue dans lsof pour travailler avec un fichier lié au noyau, souvent appelé kernel map. Dans MYLSOF, elle est acceptée pour respecter la syntaxe, mais elle ne modifie pas directement la collecte basée sur /proc.

Logique d'implémentation :

30. La fonction vérifie qu'un chemin de fichier est bien fourni après -k.
31. Elle contrôle si le fichier est lisible avec access().
32. Si le fichier est absent ou inaccessible, un avertissement est affiché.
33. Le programme continue ensuite son exécution normale, car la version MYLSOF ne dépend pas d'une table de symboles du noyau.

Exemple d'appel :

```
./mylsof -k /boot/System.map-$(uname -r)
```

Sortie attendue :

```
Avertissement : fichier -k non lisible. Option acceptée mais ignorée dans MYLSOF.
```

Explication : Dans ce projet, -k est donc une option de compatibilité. Elle montre que le programme sait reconnaître l'option, mais sans prétendre reproduire toute la complexité interne de lsof.

- **Fonction de gestion : gerer_options_MYLSOF()**

Rôle : Cette fonction est le point d'entrée logique des options. Elle analyse les arguments passés à la ligne de commande, reconnaît les options valides et active les bons champs dans la structure Options.

Logique d'implémentation :

34. Elle parcourt argv à partir de l'indice 1, car argv[0] correspond au nom du programme.
35. Elle reconnaît les options simples comme -l, -o, -t, -a et -z.
36. Elle traite les options avec paramètre comme +D <dossier> et -k <fichier>.
37. Elle appelle directement l'aide si -h est rencontré.
38. Elle retourne une erreur claire si une option inconnue est saisie.

Exemple d'appel :

```
./mylsof -a -o -z +D /tmp
```

Sortie attendue :

Options reconnues : -a, -o, -z, +D

Le programme affiche les fichiers ouverts dans /tmp avec offset et colonne zone.

Explication : Cette fonction rend le main plus propre. Au lieu de mélanger le parcours de /proc avec la lecture des arguments, elle sépare la configuration du programme et son exécution.

III. DIFFICULTÉS RENCONTRÉES

La réalisation de ce projet, bien que très enrichissante, n'a pas été sans obstacles. Nous avons rencontré plusieurs difficultés majeures durant le développement de **mylsof** :

- **Complexité du système de fichiers /proc :** La navigation dans **/proc** a été déroutante au début. Comprendre la structure précise de chaque fichier (comme **status**, **fd**, **maps**) et le formatage spécifique de leurs données a requis beaucoup de documentation et de tests.
- **Gestion de la mémoire et des chaînes de caractères en C :** Le langage C demande une gestion manuelle de la mémoire. Nous avons dû faire attention aux fuites de mémoire (memory leaks) lors de l'allocation de tampons pour lire les chemins de fichiers ou les noms de processus, ainsi qu'aux erreurs de segmentation.
- **Permissions et Liens symboliques :** Certains fichiers dans **/proc** appartiennent à root ou nécessitent des droits spécifiques. De plus, la résolution des liens symboliques (notamment pour les sockets ou les pipes) avec **readlink** a parfois posé problème car ces liens ne pointent pas vers de vrais fichiers sur le disque.
- **Synchronisation des options :** Assurer que les options de chaque membre du groupe (Cassandra, Naomy, Ange, Salif) puissent cohabiter sans conflit dans le programme principal a demandé une bonne coordination pour la gestion des arguments.

CONCLUSION

La réalisation du projet **mylsof** constitue une étape charnière dans notre apprentissage des systèmes d'exploitation. En reconstruisant intégralement une commande aussi complexe que **ls**, nous avons franchi le cap de l'utilisateur pour celui de l'architecte système. Ce projet ne se limitait pas à un exercice de programmation en C ; il était une immersion directe dans les entrailles du système d'exploitation Linux.

D'un point de vue technique, les résultats obtenus sont très satisfaisants. Nous avons réussi à mettre en œuvre un moteur capable de parcourir le système de fichiers **/proc**, d'interpréter ses structures complexes et de restituer des informations cohérentes à l'utilisateur. La modularité de notre code, divisée en quatre fichiers distincts gérés par Cassandra, Naomy, Ange et Salif, a permis de couvrir un large spectre de fonctionnalités. Que ce soit le filtrage fin par identifiants (UID, PID, GID), l'analyse des protocoles réseaux (TCP/IP, sockets Unix), ou encore l'inspection des mappings mémoire et des contextes de sécurité (SELinux), chaque module a contribué à la robustesse de l'outil final.

Ce projet nous a permis de consolider de nombreuses compétences clés :

- **Programmation système en C** : Nous avons maîtrisé les appels système critiques (`opendir`, `readdir`, `readlink`, `stat`, `snprintf`) et la gestion manuelle de la mémoire, évitant les fuites et les erreurs de segmentation.
- **Compréhension du noyau Linux** : L'exploration du répertoire **/proc** nous a enseigné comment le noyau organise et expose ses données internes. Nous avons compris la relation étroite entre les entrées de la table des descripteurs de fichiers et les inodes du système de fichiers.
- **Résolution de problèmes** : Face à des difficultés telles que les permissions refusées sur certains processus, la résolution de liens symboliques brisés ou le parsing de formats de fichiers hétérogènes, nous avons dû faire preuve de rigueur et d'ingéniosité pour adapter notre code en conséquence.

Bien que notre version de **mylsof** soit fonctionnelle et couvre un grand nombre de cas d'usage, la commande originale **ls** reste un outil colossale avec des décennies de développement. Certaines optimisations de performance et la gestion de cas de bord très rares pourraient faire l'objet de travaux futurs. Toutefois, en respectant la consigne de ne pas utiliser de bibliothèques externes et de tout coder à partir de zéro, nous avons atteint notre objectif pédagogique : comprendre « comment ça marche ».

En conclusion, ce projet a été une expérience formatrice qui a transformé des concepts abstraits de théorie des systèmes en lignes de code concrètes. Il nous a équipés d'une solide base pour aborder des sujets plus avancés tels que le pilotage de périphériques, la programmation réseau bas niveau ou la contribution au développement du noyau Linux lui-même.